SSISDE function reference

Seven modules: three for data generation and four for estimation and recovery.

Notation. S or X = state path; t = time grid; dt = time step; N = sample count; T = duration; S0 = initial state. W is simulated Brownian motion, BQ is recovered Q-noise, and BP is recovered physical Brownian motion. Trailing underscores mark fine-grid arrays. Tuples below follow the code's return order.

Data generation

data_generation/simulation.py

generate_noise(N, T, seed) returns **(t, dt, dW, W)**: time grid, step, Brownian increments and path. **simulate(mu, sigma, N, T, S0, seed)** takes callable coefficients and returns **(t, dt, S, W, mu(S), sigma(S))** using Euler-Maruyama.

data_generation/processes.py

mu(x, drift, ...) and **sigma(x, diff, ...)** evaluate a named process's coefficients. **simulate_process(dW, dt, S0, ...)** accepts existing Brownian increments and returns **(S_, mu_exact_, sigma_exact_)**. Black-Scholes uses drift='BS', diff='BS', r and sigm.

generate_constant(N, T, S0, seed, r, sigm), **generate_quadratic(N, T, S0, seed)**, and **generate_ou(N, T, S0, seed, kappa, theta, sigm)** each return the six outputs of simulate. Their defaults preserve the experiment seeds (31, 2 and 3 respectively).

constant_drift / constant_diffusion: r and sigm. **quadratic_drift / quadratic_diffusion**: $0.3x^2 + 0.7x$ and $0.1x^2 + 0.5x$. **ou_drift / ou_diffusion**: kappa(theta - x) and sigm. Each takes x, with optional coefficients where applicable.

data_generation/coarse_graining.py

coarse_grain(S_, BQ_, W_, mu_exact_, sigma_exact_, sigma_pred_, M, K) returns **(sigma_pred, mu_exact, S, BQ, W, sigma_exact)**, taking every M-th entry for K samples.

Diffusion estimation

models/diffusion.py

compute_quadratic_variation(W) returns the sum of squared increments. **estimate_quadratic_variation(S_, t)** returns **(QV, dQV, diff_method)**: cumulative variation, its time derivative and the finite-difference object.

fit_diffusion_model(S_, t, dQV, ...) and **fit_quadratic_diffusion(S_, t, dQV, ...)** return fitted PySINDy variance models. Both use degree 4; only the first includes a constant term. Optional settings are threshold, nu, max_iter and tol.

diffusion_model(threshold, nu, max_iter, tol) returns an unfitted variance model. **fit_diffusion(S, t, dQV, thresholds, nus, max_iter, tol)** selects settings on the last 20% of the path and returns **(model, best, results)**: fitted model, selected settings and trial records.

diffusion_derivatives(sigma_pred_, S_, diff_method) and **coarse_diffusion_derivatives(sigma_pred, S, diff_method)** return the first, second and third diffusion derivatives with respect to the state. Regression fits sigma^2; take its positive square root before computing these derivatives.

Drift, noise and identification

The remaining three model modules

models/drift.py

fit_signature(S_, BQ_, t, keep_cols, M) returns **l21**, one fitted signature coefficient per window. keep_cols selects signature columns and M sets the window length.

drift_coefficients(mu_exact, sigma_pred, sigma_prime, sigma_2prime, l21) returns **(mu_0, a, b, b_)**: initial drift and recovery coefficients. Only mu_exact[0] is used.

recover_drift(S, a, b, b_, mu_0) returns **(mu_pred1, mu_pred2)** for Methods 1 and 2. **recover_drift_ode2(S, sigma_pred, sigma_prime, sigma_2prime, mu_0, lambda_0)** returns **(mu_pred11, lamda_pred)**: drift and spatial slope. Supply the initial drift and slope explicitly.

models/noise.py

recover_q_noise(S_, sigma_pred_) returns the fine-grid Q-noise path **BQ_**. **recover_p_noise(BQ, mu_pred1, sigma_pred, T)** returns **(BP, dt, t)**: physical noise, coarse time step and grid.

recover_p_noise_method2(BQ, mu_pred2, sigma_pred, T) returns the same tuple using Method 2 drift. **recover_noise(BQ, drift, diffusion, dt)** accepts a time step directly and returns only the physical noise path.

models/stochastic_sindy.py

StochSINDy(...) creates an identifier. Inputs set polynomial degrees (mu_deg, si_deg), penalty (pen), cross-validation (use_cv, k, alphas, rhos, use_1se), and thresholds (thr1, thr2).

fit(X, BP, dt, alpha_mu, alpha_si) returns the fitted model. **mu(x)** and **si(x)** predict drift and diffusion; **show()** prints equations and errors. **plot_cv(output_prefix)**, **plot_size(count_bias, output_prefix)** and **plot_results(X, BP, BQ, W, mu_true, si_true, output_prefix, drift_title)** display diagnostics and save plots where implemented.

stochastic_sindy_candidates() returns optional tuning configurations. **tune_stochastic_sindy(X, BP, dt, train_fraction, candidates)** selects settings using held-out increment error and returns **(tuned_model, baseline_model, selection, records)**. True drift and diffusion are not inputs to selection.

Example imports

```
from data_generation.processes import generate_ou
from models.drift import recover_drift, recover_drift_ode2
from models.noise import recover_p_noise
from models.stochastic_sindy import StochSINDy
```

Optional arguments retain their source defaults. The consolidation changes module locations; the four experiment notebooks retain their numerical settings and formulas.